# PorkSort: Pass/Fail Inspection of Pork Rasher Packaging

Oct 25, 2026 · Hashim Navas

Hashim Navas, student ID [add] · MMA3001 Numerical Methods and Machine Learning, Semester 2 2026, Monash University · Individual project, Dataset 3 · Repository: github.com/hashim-edu/mma3001-project-pork_sort

## Summary

PorkSort inspects overhead camera frames of sealed pork rasher trays on a packing line. For each frame it returns PASS or FAIL with the defects found and where they are, so that suspect packs can be sent for manual re-inspection. The selected model is a YOLOv8n detector, fine-tuned on 869 distinct frames and run with ONNX Runtime on a laptop CPU, whose defect score is compared with a threshold chosen to minimise expected cost. Because the provided test split shares trays with the training data, I evaluated on a blocked split with a 30-frame buffer. On its 174 test frames the detector's ROC AUC is 0.887 (95% CI 0.810 to 0.947), 0.148 above the best classical model (paired 95% CI 0.068 to 0.235). It does not meet my acceptance criteria: when a missed defect costs ten re-checks, no model does materially better than re-checking every pack. When a miss costs one re-check, the detector passes 56% of good frames and lets 3.6% of defective frames through. It takes 41 ms per frame on an Apple M4 CPU.

## 1 Engineering problem

Pork rashers are packed in plastic trays and sealed with lidding film on automated tray-sealing lines, which run at about 120 packs per minute [1] and up to 200 on recent machines [2]. An unsealed or damaged pack risks spoilage and a product recall, and loose or twisted meat leads to customer complaints. At two to three packs per second, people cannot inspect every pack consistently over a shift, so automating the check is a machine-vision problem within the instrumentation and control of the line.

The Dataset 3 brief poses an open problem and asks for a narrower, achievable one. I chose:

> *Given one RGB frame from the overhead camera, decide whether the visible tray should PASS or FAIL packaging inspection, and show where any defect is.*

The intended use is decision support. Every FAIL frame is shown to an operator with its defect boxes, and the operator makes the final call; the system never rejects product on its own. The scope is limited by what the data can support:

- **Per frame, not per tray.** Frames carry no tray identifier, and a tray stays in view for about 12 frames, so a tray-level decision would need tracking.
- **Packaging and placement only.** The labels cover five defects (unsealed film, other packaging errors, loose meat, twisted meat and wrinkles) and nothing about meat quality.
- **One line, one camera, one lighting setup.** No claim is made about other lines.

FAIL means any of the five defects is present. All five are treated as equally serious, because the safe response to an unconfirmed defect is a human check.

## 2 Inputs, outputs and data

### 2.1 Top-level inputs and outputs

| Item | Format and units | Domain | Meaning and use |
|---|---|---|---|
| Input frame | JPEG, PNG or BMP; RGB, 8 bits per channel | 720 x 540 px overhead conveyor frame. Other sizes are letterboxed and flagged | One camera frame of the line |
| Defect score s | Dimensionless | 0 to 1 | Highest confidence among detected defect boxes; ranks how suspicious the frame is |
| Threshold τ | Dimensionless | 0 to 1, fixed from validation data for a cost ratio r | The frame FAILs when $s \geq \tau$ |
| Status | Text | PASS, FAIL or ERROR | Operator action: let through, re-inspect, or check the camera or file |
| Defects | Class; confidence 0 to 1; box corners in px; area as a fraction of the frame | Five classes | What is wrong and where to look |
| Timing | Milliseconds | Positive | Load, preprocess, inference, postprocess and total time; checks the line budget |
| Warnings | Text | Wrong size, very dark (mean below 15/255), very bright (above 240/255), near-uniform | The input is unlike the training data |

Each frame produces a JSON report and an annotated PNG, and a batch adds a summary and an HTML index. Invalid inputs never stop a batch: a file that is not an image, or cannot be decoded, gets status ERROR with the reason, and grey or RGBA images are converted to RGB. The command-line tool exits with code 2 if any frame was an ERROR and 1 for invalid arguments, which a test checks with a mix of good, corrupt and non-image files. Appendix B gives the inputs, outputs and errors of the main functions, and the HTML documentation collects every public function's docstring.

### 2.2 The data

The data is the Roboflow "Pork Rasher Error(Packaging)" export, version 4, under CC BY 4.0 [3]: 3,549 images from 1,272 distinct frames of one overhead conveyor video, with bounding boxes for the five defect classes. Frames were already resized to 720 x 540 px by centre crop, and each training frame has three copies with exposure shifted by up to ±10%; pixel values are normalised inside the pipeline. Labelling a frame FAIL if it has any defect box makes PASS the minority, about 20% of images. Twisted meat, loose meat and wrinkle each appear in fewer than 70 training images, and wrinkle in no test frame. Defect sizes differ greatly: the median packaging-error box covers 0.38% of the frame (about 37 x 37 px), the median unsealed box 10.9%.

### 2.3 Leakage in the provided split, and the blocked split

The provided test frames are interleaved with training frames from the same video: all 80 have a training frame within 30 frame numbers, and 23 within one (Appendix C). Neighbouring frames show the same tray slightly moved, so testing on them partly tests on trays seen in training, a form of leakage [4]. The correlation between frame thumbnails repeats every 11.9 frames, a period found from the FFT of the similarity-against-gap curve. That is the spacing of trays on the belt, so frames 30 or more apart, about 2.5 tray spacings, show different trays.

The blocked split assigns whole blocks of 200 consecutive frame numbers to train, validation or test, as recommended for temporally structured data [5], and drops any frame within 30 frame numbers of a higher-priority split. Among 500 random block assignments I kept the one whose validation and test PASS rate and class mix were closest to the whole dataset, using labels only, never model results. The result is 869 training frames (2,433 images with copies), 192 validation and 174 test frames, of which 37 and 34 are PASS. Validation and test use one copy of each frame.

## 3 Computational approach

Every model, simple or deep, is reduced to one defect score s in [0, 1] per frame, and the frame FAILs when $s \geq \tau$. Every model is then judged by the same rule and the same threshold procedure (Figure 1).

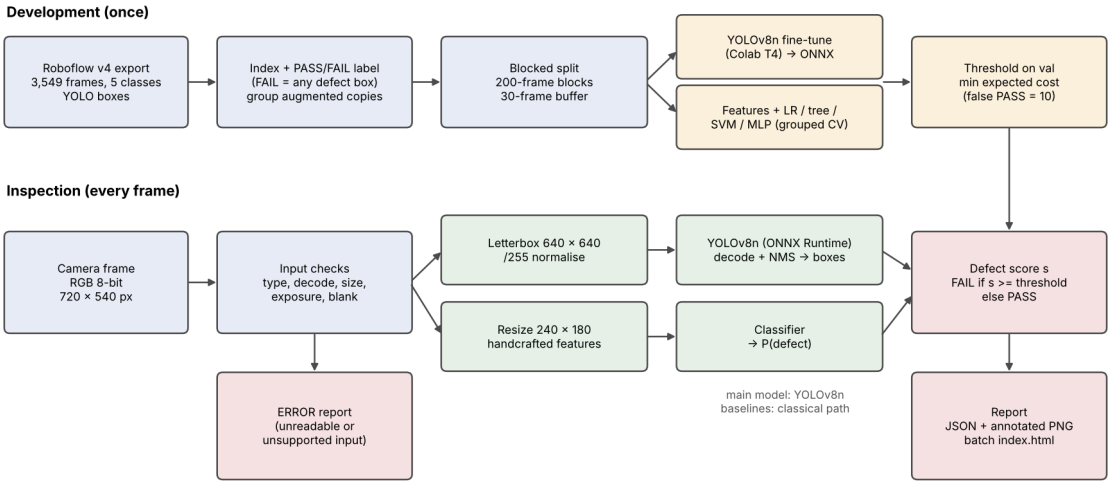



*Figure 1. PorkSort architecture. Development (top) runs once; inspection (bottom) runs on every frame. The classical path is the baseline family the selected YOLOv8n detector was compared against.*

### 3.1 Classical models on handcrafted features

A frame has over a million pixel values and there are only 869 distinct training frames, so the classical models work on features. Each frame is resized to 240 x 180 px and summarised by 1,439 features in six groups: a joint HSV colour histogram; the share, centroid and spread of meat-coloured pixels; glare on a 4 x 3 grid; Sobel edge strength on a 6 x 4 grid; a local binary pattern texture histogram [6]; and a histogram of oriented gradients (HOG) [7]. The edge and HOG features rest on finite-difference estimates of the image gradient.

Four classifiers, the classification counterparts of regression models from the unit [8], were trained on these features: logistic regression, a decision tree, a support vector machine (SVM) with an RBF kernel and a multilayer perceptron (MLP), using scikit-learn [9] and scikit-image [10]. Features were standardised for the logistic regression, SVM and MLP; otherwise large-range features dominate the SVM's distances and make gradient descent badly conditioned. Classes were weighted by inverse frequency so the PASS minority was not ignored. Hyperparameters were chosen by five-fold cross-validation on the training set, scored by ROC AUC, with folds grouped by 200-frame block so the copies of one frame never sat on both sides of a fold (settings in Appendix D).

### 3.2 YOLOv8n detector

The smallest defects are about 37 px wide, and Section 4 shows that global features cannot see them, so a detector that examines every part of the image is the natural tool. I chose YOLOv8n, the smallest YOLOv8 model [11], [12]: it is fast enough for a CPU on the line, its pretrained weights transfer to a small dataset, and its boxes explain each FAIL to the operator. A convolutional backbone produces feature maps at three scales, and a head predicts a box and five class scores at each of 8,400 grid positions for a 640 px input. Boxes below a confidence threshold are dropped, and non-maximum suppression (NMS) keeps the highest-scoring box and deletes any box of the same class whose intersection over union (IoU) with it exceeds 0.6.

I fine-tuned COCO-pretrained weights at a 640 px input, with frames letterboxed to 640 x 480 plus grey padding: up to 60 epochs, early stopping after 20 without improvement, batch 16, seed 0 and default augmentation, on a free Colab T4 GPU. Training stopped at epoch 39 after 22 minutes, keeping the epoch-19 weights, the best on validation mAP. The network was exported to ONNX and runs with ONNX Runtime on a CPU [13]. Letterboxing, output decoding and NMS are implemented in NumPy within the

package, so inspection needs neither PyTorch nor a GPU. The deployed model runs at a 480 px input (Section 5.3).

### 3.3 Choosing the threshold by cost

A threshold of 0.5 has no special meaning here, because the two errors have different costs: a defective pack that passes risks a recall, while a good pack that fails costs a re-check. I chose τ to minimise the expected cost per frame on the validation set:

$$C(\tau) = [\ r \cdot FN(\tau) + FP(\tau)\ ] / N$$

FN counts defective frames passed, FP good frames failed, and N all frames. I set the cost ratio r = 10, so a shipped defect counts as ten re-checks; Section 5.4 tests this choice. The candidate thresholds are the midpoints between consecutive distinct scores, so the search finds the exact minimum on the data, and ties go to the lower threshold because a false PASS is the more expensive mistake.

The method assumes that the annotations are correct and complete for what is visible, that new frames come from the same line, camera and lighting, that the five defect classes are equally serious, and that each frame can be judged on its own. Its known limitations are in Section 6.

## 4 Validation and verification

### 4.1 What was validated and verified

Validation asks whether the decisions are right: the PASS/FAIL decision for each of the 174 withheld frames of the blocked test split was compared with its label, once, after every choice had been fixed. Verification asks whether the code does what was intended. The repository has 52 automated tests (Appendix E), four of which compare the code with answers known in advance: the YOLO decoder recovers a hand-encoded box to within 1e-5, the FLOP formula matches a direct loop count, the threshold search matches a 20,001-point brute-force search, and the optimised histogram is bit-identical to NumPy's.

I also compared the ONNX pipeline with an established implementation, Ultralytics' own predictions, on 60 test frames. Of the 41 boxes Ultralytics found at confidence 0.25 or more, 34 matched at IoU ≥ 0.9 (median 0.94), and frame scores differed by 0.04 on average. Two experiments placed the cause in the input rather than the decoding: switching to OpenCV's resizing barely changed the difference (0.0392 to 0.0389), while moving the padding from centred to top-aligned shifted scores by 0.024. Ultralytics feeds its PyTorch model a 640 x 480 rectangle, but the exported model needs a padded square. All results below use the deployed ONNX pipeline.

### 4.2 Why this method is appropriate, and the acceptance criteria

Withheld data is the only direct test on unseen frames, and the blocked split keeps every test tray out of training. FAIL is the positive class. The primary metric is the **escape rate**, FN / (TP + FN), the share of defective frames passed; the secondary metric is the **false reject rate**, FP / (FP + TN), the share of good frames re-checked. ROC AUC, the area under the ROC curve computed by the trapezoidal rule, measures how well a model ranks frames across all thresholds. Accuracy is not used, because always answering FAIL scores 80.5%. Rates from 34 PASS frames are uncertain, so each has a Wilson score 95% interval [14], which unlike the normal approximation stays valid at 0% and 100%; AUC intervals come from 2,000 bootstrap resamples. Before seeing any detector results I set two acceptance criteria: an escape rate with an upper 95% bound of at most 5%, and a false reject rate of at most 30%.

Validation did influence model selection: grouped cross-validation chose the classical hyperparameters, and the validation set chose the threshold, the input size and the final model. The test set was used once, at the end.

### 4.3 Results

*Table 1. Blocked test set (174 frames, 34 PASS), thresholds chosen on validation at r = 10. Intervals are 95%.*

| Model | ROC AUC (95% CI) | Threshold | Escape rate | False reject rate | Cost per frame |
|---|---|---|---|---|---|
| Always FAIL (baseline) | 0.500 | 0 | 0.000 | 1.000 | 0.195 |
| Colour-histogram logistic regression | 0.739 (0.652 to 0.822) | 0.205 | 0.000 | 0.941 | 0.184 |
| Logistic regression | 0.739 (0.649 to 0.823) | 0 | 0.000 | 1.000 | 0.195 |
| Decision tree | 0.717 (0.621 to 0.807) | 0.147 | 0.029 | 0.971 | 0.420 |
| SVM (RBF) | 0.724 (0.633 to 0.806) | 0 | 0.000 | 1.000 | 0.195 |
| MLP | 0.717 (0.618 to 0.805) | 0 | 0.000 | 1.000 | 0.195 |
| YOLOv8n, 640 px input | 0.859 (0.773 to 0.926) | 0.004 | 0.000 | 1.000 | 0.195 |
| **YOLOv8n, 480 px input (selected)** | **0.887 (0.810 to 0.947)** | 0.007 | 0.000 | 1.000 | 0.195 |

YOLOv8n ranks frames far better than any classical model (Figure 2). On the same test frames its AUC beats logistic regression's by 0.148, with a paired bootstrap 95% interval of 0.068 to 0.235, so the gain is not sampling noise. Scored against the PASS frames, its AUC is 0.95 for frames with unsealed film, 0.84 for packaging errors, 0.71 for loose meat (6 frames) and 0.97 for twisted meat (4 frames); logistic regression's is 0.86, 0.67, 0.51 and 0.49. Global features see the large, tray-sized defect but not the small ones.

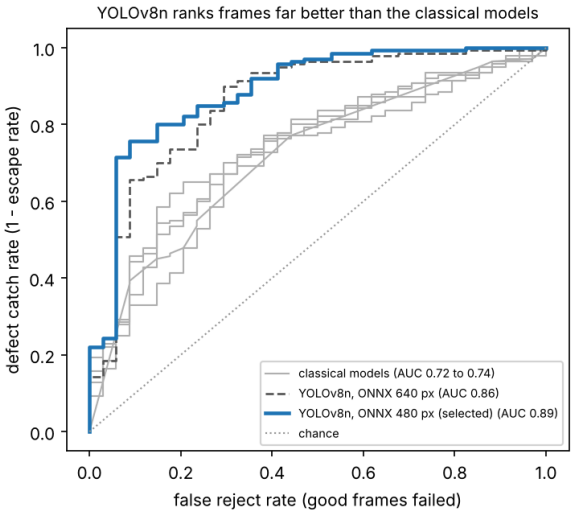



*Figure 2. ROC curves on the blocked test set. The selected detector (blue) dominates every classical model (grey).*

Ranking well is not enough at r = 10. With 80% of frames defective, passing a good frame saves one re-check while each escape costs ten, and the lowest-scoring validation frames still include defective ones, among them five wrinkle frames, a class with only three training frames. The detector's cost-optimal threshold therefore fails every frame. The best any model does is the colour-histogram model, which passes 2 of the 34 good frames, and no model meets the acceptance criteria. Table 2 shows where the detector does pay off.

*Table 2. Selected detector: operating points chosen on validation, evaluated on test.*

| Cost ratio r | Threshold | Escape rate (95% CI) | False reject rate (95% CI) | Cost per frame (re-check all: 0.195) |
|---|---|---|---|---|
| 1 | 0.128 | 3.6% (1.5% to 8.1%) | 44% (29% to 61%) | 0.115 |
| 2 to 50 | 0.007 | 0% | 100% | 0.195 |

At r = 1 the detector passes 19 of the 34 good frames and lets 5 of the 140 defective frames through, cutting the cost per frame by 41% (Figure 3). Its main failure modes are small packaging errors at tray corners and loose pieces at the frame edge, which make up most escapes. Several false rejects are trays whose film looks lifted but whose frame carries no label, which may be label ambiguity rather than model error.

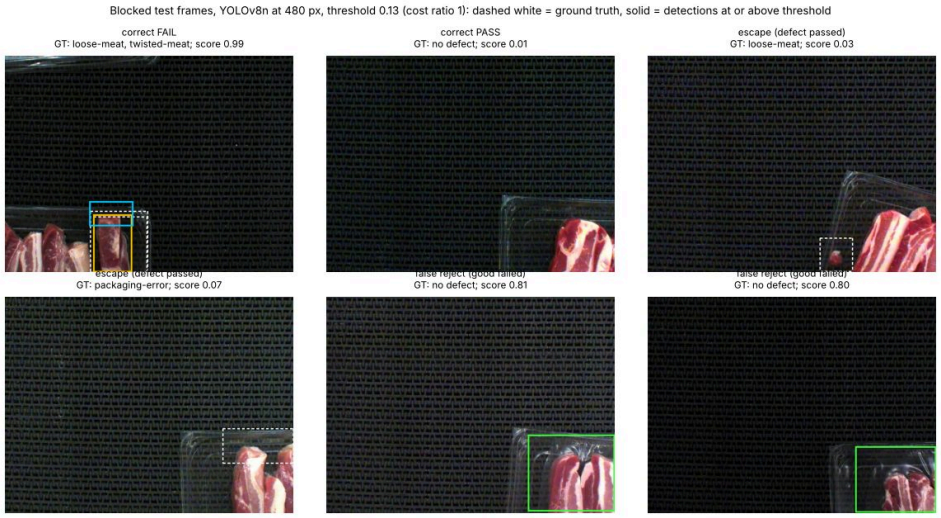



*Figure 3. Blocked test frames at the r = 1 threshold. Dashed white boxes are labels; solid boxes are detections.*

The split matters as much as the model. On the leaky Roboflow test split, logistic regression scores an AUC of 0.808 and the SVM 0.826, against 0.739 and 0.724 on the blocked split, so a random split would have overstated them by 0.07 to 0.10. I did not train a second detector on the Roboflow split to measure the same effect for YOLO.

### 4.4 What the validation cannot establish

It cannot establish performance on another line, camera, lighting setup or product; behaviour per tray, since a defect visible in one frame may be out of view in the next; recall for the rare classes, since wrinkle has no test frames and loose and twisted meat four to six; the effect of label errors; or whether r = 10 reflects the plant's real costs.

## 5 Alternatives, optimisation and performance

### 5.1 Comparison of alternatives

*Table 3. Alternatives compared on the brief's criteria. Times are per frame on the M4 laptop CPU unless marked.*

| Model | Blocked test AUC | Time per frame | Model size | Interpretability | Localises defects |
|---|---|---|---|---|---|
| Logistic regression | 0.739 | 2.8 ms features (optimised) + 0.25 ms (container) | about 35 kB | High: one weight per feature | No |
| Decision tree (depth 5) | 0.717 | features + 0.11 ms (container) | under 10 kB | High: readable rules | No |
| SVM (RBF) | 0.724 | features + 2.0 ms (container) | 1,606 support vectors | Low | No |
| MLP (64 hidden units) | 0.717 | features + 0.31 ms (container) | about 0.75 MB | Low | No |
| **YOLOv8n, 480 px** | **0.887** | 36.7 ms inference, 41 ms end to end | 12.7 MB (fp32), 6.4 MB (fp16) | Medium: boxes show why | Yes |

I selected YOLOv8n at 480 px. It has the highest AUC, with a paired interval that excludes zero; it is the only model that shows the operator where the defect is; and it is fast enough for the line (Section 5.4). Its costs are a larger model, GPU training and less transparent internals. It is selected as the best model available, not as one that meets the acceptance criteria.

### 5.2 Profiling and optimising the classical pipeline

Feature extraction, not the classifier, dominates the classical pipeline's time. Profiling `features.extract` with line_profiler on the M4 put RGB-to-HSV conversion at 36% of the time, the 3D colour histogram at 14%, texture at 13%, HOG at 11% and resizing at 10%. Four changes followed, each checked for its numerical effect:

1. PIL's 8-bit conversion, written in C, replaced scikit-image's float64 HSV conversion. Values differ from the reference by at most 1/255, far finer than the histogram bins of 1/8 and 1/4, and a test checks this bound.
2. The histogram computes each pixel's bin directly and counts with `bincount`; a test confirms the counts are bit-identical to NumPy's `histogramdd`.
3. Dropping one feature group at a time showed that only colour and HOG matter: without them validation AUC fell by 0.026 and 0.036, while each other group changed it by under 0.003, so the other four were removed.
4. Resizing is 2D interpolation. Across three working sizes and four interpolation methods (Appendix C), box (area-averaging) filtering was best at every size: nearest-neighbour keeps one pixel in several, so the belt's fine weave aliases into noise, while box filtering averages it out. I chose box filtering at 120 x 90 px.

Extraction time fell from 13.9 ms to 9.0, 4.9 and finally 2.8 ms per frame on the M4, 4.9 times faster in total, while validation AUC rose from 0.718 to 0.758 and test AUC from 0.739 to 0.748.

### 5.3 YOLOv8n: FLOPs, roofline, precision and input size

Counting each convolution in the ONNX graph as 2 x C_out x H_out x W_out x (C_in / groups) x k² gives 8.64 GFLOPs per 640 px frame, within 5% of the 8.2 GFLOPs Ultralytics reports. If each layer reads its weights and input and writes its output once, the network moves 153 MB per frame, an arithmetic intensity of 57 FLOP/byte (layers range from 0.5 to 174). For the roofline model [15], the unit's formula (cores x clock x vector lanes x FMA factor x pipes) gives the M4's four performance cores a peak of 4 x 4.4 GHz x 4 x 2 x 4 ≈ 563 GFLOP/s [16], where four pipes per core is an estimate. With about 120 GB/s of bandwidth the M4's machine balance is 4.7 FLOP/byte, and the Colab T4's about 25 (8.1 TFLOP/s, 320 GB/s) [17], so the network is compute-bound on both (Figure 4). On the M4 it reached 140 GFLOP/s, a quarter of the estimated roof, probably because the virtual machine's cores may not all be performance cores, small layers use the vector units poorly, and the count omits activations, concatenation and decoding.

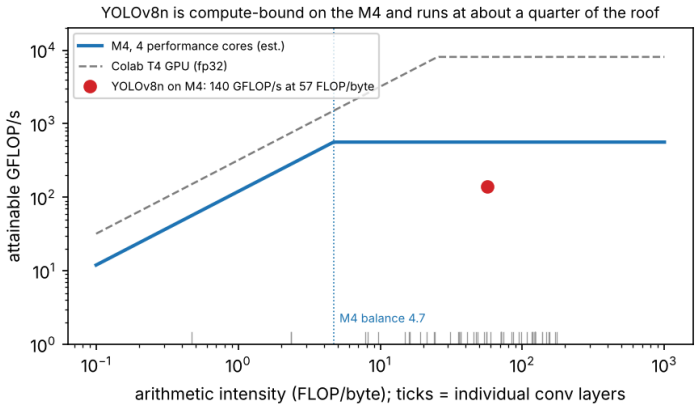



*Figure 4. Roofline for the M4 and the T4. Ticks mark individual convolution layers; the red point is the measured whole-network performance on the M4.*

**Half precision.** Exporting the network in fp16 halved the file from 12.8 to 6.4 MB and cut 640 px inference on the M4 from 61.9 to 42.9 ms, because an ARM vector register holds eight fp16 values instead of four fp32. Over 60 test frames the largest change in a frame score was 0.0008, consistent with fp16's unit roundoff of 2^-11 ≈ 0.0005; no decision changed and test AUC was identical. fp16 was exported only at 640 px, so the deployed 480 px model is fp32, and combining the two is an untested further saving.

**Threads and input size.** Inference at 640 px took 207 ms on one thread and 62 ms on four, a 3.3 times speed-up. Run at 320, 480, 640 and 800 px (Appendix C), the network trained at 640 px gave validation AUCs of 0.848, 0.875, 0.870 and 0.851, while time grew roughly with pixel count: 17.7, 36.7, 61.9 and 98.5 ms. I chose 480 px on validation, matching 640 px within noise at 59% of the time; its higher test

AUC (0.887 against 0.859) is not claimed as a real gain. Running at 800 px did not help the small packaging errors, which points to training at a higher resolution.

### 5.4 Line budget, sensitivity and repeatability

A line running 120 to 200 packs per minute leaves 300 to 500 ms per pack if one frame per pack is judged, so at 41 ms end to end the system is 7 to 12 times inside that budget. Judging every frame, at 11.9 frames per tray, leaves 25 to 42 ms: the 480 px model fits only at 120 packs per minute, while the 320 px model (21 ms) fits at both speeds for 0.027 less validation AUC.

Sweeping r from 1 to 50 (Appendix C) moves the detector's operating point only between r = 1 and r = 2, and the classical models' thresholds do not move at all, so their weakness lies in ranking, not in the choice of r. Over five random seeds the MLP's test AUC was 0.716 ± 0.008, converging in 50 to 70 iterations under early stopping. YOLOv8n was trained once.

## 6 Limitations and lessons learned

### 6.1 Limitations

- **It does not meet the acceptance criteria.** At r = 10 the detector is a prioritisation aid that shows boxes, not an automatic gate.
- **Per-frame decisions.** A tray is in view for about 12 frames and is often only partly visible, so a frame can be labelled PASS while its defect is just out of view. A deployed system should track each tray and fail it if any of its frames fails.
- **Rare classes.** Wrinkle has three training frames and no test frames, so nothing is claimed about its recall.
- **One recording.** All frames come from one line with an unknown frame rate, and 32 frame names carry two different images, which suggests two recording sessions.
- **One training run.** The detector's run-to-run variation is unknown.

### 6.2 Lessons learned

- **The split decides what is measured.** The provided split overstated classical AUC by up to 0.10, because test trays had been seen in training.
- **Ranking is not deciding.** An AUC of 0.89 still fails a 10 to 1 cost test when 80% of frames are defective, so a model must be judged at its operating point and cost.
- **Match the method to the scale of the defect.** Global features found unsealed trays but not small local defects, which the detector found.
- **Profile before optimising.** The slowest step was colour conversion, not the model.

The next steps are to train at a higher input resolution, such as 960 px, for the small packaging errors behind most escapes; to collect more wrinkle, loose-meat and twisted-meat examples or merge them into one placement class; to track trays across frames; to train with several seeds; and to replace r = 10 with costs measured at the plant.

## 7 AI use and reflection

**Tools and tasks.** I used Claude (Anthropic) through the Claude app, linked to my laptop and a cloud workspace. It wrote most of the code (the package, analysis scripts, tests and Colab training notebook), ran the data audit and analyses, and drafted this report against the brief and rubric. Google Colab provided the GPU for training.

**Level of contribution and why.** High: I estimate the AI produced about [confirm]% of the code and the first draft of the report text. I used it because the project needed tools I had not used before (YOLO, ONNX, profiling image code) and fell in a week with two other tests.

**How I directed it.** I chose the dataset and the per-frame PASS/FAIL goal with defect locations, required the work to use methods from the unit outline and to run on my MacBook without a GPU, and chose to

train on Colab, where I ran the training and reported its errors back. The AI proposed the blocked split, the cost-based threshold, r = 10 and the acceptance criteria; I reviewed and accepted each one and am responsible for them. When I checked the draft report against the brief, I found it over the page limit and had it rechecked against every criterion and shortened.

**How the output was checked.** By 52 automated tests, four against answers known in advance; by an independent comparison with Ultralytics' predictions, with two experiments to explain the differences; by looking at sample frames and every figure; and by tracing each number in this report to a file in `results/`.

**Errors and unhelpful output.**

- Its first frame-similarity analysis used a ±2 standard deviation band too wide to show anything; the plot showed a repeating pattern instead, and an FFT of the curve gave the 11.9-frame tray spacing.
- Its Colab notebook failed on first run because the code folder had the same name as the Python package and hid it, and it once delivered an out-of-date copy of the code to my folder, caught by checking file sizes.
- Two of its tests failed because of bugs in the tests themselves. Draft text assumed a 30 fps camera the data does not support, and claimed that no model beat re-checking every pack at r = 10, when the colour-histogram model passes 2 good frames.
- Its first full draft was over the page limit and did not document function-level inputs and outputs.

**Effect on my learning.** The AI made a far more thorough project possible in the time available. The risk is understanding the result less well than if I had built it myself. To reduce that, [confirm: e.g. I worked through the split design, the cost threshold, NMS and the roofline with guided explanations until I could explain them without notes].

## References


1. "Proseal launch for GT2X Twin E-seal," *BakeryandSnacks.com*, Jul. 2012. [Online]. Available: bakeryandsnacks.com
2. "Harpak ULMA launches high speed grind line tray sealing application powered by Mondini Sinfonia," *Business Wire*, Jan. 2026. [Online]. Available: businesswire.com
3. "Pork Rasher Error(Packaging)," version 4, Roboflow Universe, workspace hello-2aqe0, licensed CC BY 4.0, 2025. [Online]. Available: universe.roboflow.com
4. S. Kaufman, S. Rosset, C. Perlich and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, 2012.
5. D. R. Roberts *et al.*, "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913-929, 2017.
6. T. Ojala, M. Pietikäinen and T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 971-987, 2002.
7. N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE CVPR*, 2005, pp. 886-893.
8. MMA3001 teaching team, *MMA3001 Numerical Methods and Machine Learning unit notes*, Weeks 2.1, 2.2, 3.5 and 5.1 to 5.6, Monash University, 2026.
9. F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011.
10. S. van der Walt *et al.*, "scikit-image: Image processing in Python," *PeerJ*, vol. 2, e453, 2014.
11. J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE CVPR*, 2016, pp. 779-788.
12. G. Jocher, A. Chaurasia and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: github.com/ultralytics/ultralytics
13. ONNX Runtime developers, "ONNX Runtime," Microsoft. [Online]. Available: onnxruntime.ai
14. E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209-212, 1927.
15. S. Williams, A. Waterman and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65-76, 2009.
16. "Apple Silicon: M4 chip specs," *Low End Mac*, 2025. [Online]. Available: lowendmac.com
17. NVIDIA, "NVIDIA T4 Tensor Core GPU." [Online]. Available: nvidia.com


## Appendix A: Repository guide

Repository: github.com/hashim-edu/mma3001-project-pork_sort. The README explains the purpose, inputs and outputs, installation, and how to reproduce every result: install with `pip install -r requirements.txt` then `pip install -e .`, unzip the Roboflow export to `data/pork_v4/`, and run `01_data_audit.py`, `02_classical_models.py`, the Colab notebook `03_train_yolo_colab.ipynb`, `04_evaluate_and_compare.py`, `05_performance.py` and `06_figures.py` in order. The inspection tool runs with `python -m porksort inspect <frames> --out reports/`; `--cost-ratio 1` selects the threshold chosen on validation for r = 1. Every public function has a Google-style docstring giving its arguments, return value and errors (collected in the HTML documentation), and the code is formatted and linted with ruff.

| Folder | Contents |
|---|---|
| `src/porksort/` | Package: dataset, splits, features, models, decision, evaluate, detector, perf, report, cli |
| `scripts/` | Numbered analysis scripts that write to `results/` |
| `notebooks/` | Colab training notebook |
| `splits/` | The committed blocked split, so every machine uses the same frames |
| `results/` | Metrics (JSON, CSV), profiles and figures used in this report |
| `tests/` | pytest suite on synthetic data; needs no dataset |
| `docs/` | HTML API documentation (pdoc, Google-style docstrings) and the pytest HTML test report |

## Appendix B: Inputs and outputs of the main functions

Boxes are `Box` records: class name, centre x and y, width and height as fractions of the image in [0, 1], and confidence in [0, 1]. Full docstrings are in `docs/api/`.

| Function | Inputs | Output | Invalid input |
|---|---|---|---|
| `dataset.parse_label_file(path)` | YOLO label file; one line per box: class id, cx, cy, w, h in [0, 1] | List of `Box` | `FileNotFoundError`; `ValueError` naming the file and line for a wrong field count, unknown class or coordinate outside [0, 1] |
| `dataset.build_index(root)` | Export folder with train, valid and test images and labels | DataFrame, one row per image: path, provided split, source frame, frame number, boxes, FAIL label | `FileNotFoundError` if the folders or a label file are missing |
| `splits.blocked_split(index, cfg)` | Image index; `BlockedSplitConfig` (block 200 frames, gap 30, validation and test fractions 0.15, 500 trials, seed 0) | Each source frame assigned to train, val, test or buffer | `ValueError` if the fractions leave no training set |
| `features.load_rgb(path)` | Image file: .jpg, .jpeg, .png or .bmp | uint8 array (H, W, 3); grey and RGBA converted to RGB | `FileNotFoundError`; `ValueError` if unsupported or undecodable |
| `features.extract(rgb, cfg)` | uint8 RGB array of any size; `FeatureConfig` (working size, interpolation, feature groups, HSV method) | float vector of fixed length for a config: 1,439 for all groups, 1,388 for colour and HOG | `ValueError` if not (H, W, 3) |
| `models.make_model(name)` | `majority`, `colour_lr`, `logreg`, `tree`, `svm` or `mlp` | Untrained scikit-learn pipeline and its hyperparameter grid | `ValueError` for an unknown name |
| `models.defect_score(model, X)` | Fitted pipeline; feature matrix (n, d) | n defect scores in [0, 1] | |
| `detector.letterbox(rgb, size, fill)` | uint8 RGB frame; square side in px; padding grey level (114) | Padded (size, size, 3) array, scale r, padding (dx, dy) in px | |
| `detector.decode(output, r, pad, img_wh, conf_thr, iou_thr)` | Raw network output (1, 9, anchors): cx, cy, w, h in letterboxed px and five class scores in [0, 1]; r and pad from `letterbox`; frame size; thresholds | Up to 300 `Box` in normalised frame coordinates, after per-class NMS | |
| `detector.nms(boxes, scores, iou_thr)` | Boxes (N, 4) as corner coordinates; scores (N,); IoU threshold | Indices of kept boxes, highest score first | |
| `detector.OnnxYoloDetector(path, conf, iou, threads).detect(rgb)` | ONNX model file; uint8 RGB frame | List of `Box`; per-stage times in ms in `last_timing` | `FileNotFoundError` if the model is missing |
| `decision.decide(boxes, threshold)` | Detected boxes; τ in [0, 1] | `Decision`: FAIL if the frame score, the highest defect confidence (0 if none), is at least τ; the score; the defects | `ValueError` if τ is outside [0, 1] |
| `decision.choose_threshold(y_fail, scores, cost_ratio)` | Labels (True = defective), scores in [0, 1], cost ratio r | (τ, minimum expected cost per frame), ties to the lower τ | |
| `evaluate.wilson_ci(k, n)` | Count k of n trials | 95% interval (lower, upper) in [0, 1] | |

| Function | Inputs | Output | Invalid input |
|---|---|---|---|
| `evaluate.passfail_metrics(y_fail, scores, threshold, cost_ratio)` | Labels, scores, τ, r | Dict: confusion counts, escape and false reject rates with intervals, expected cost, ROC AUC | |
| `evaluate.bootstrap_auc(y_fail, scores, n_boot, seed)` | Labels, scores, resamples (2,000), seed | Percentile 95% interval for ROC AUC | |
| `perf.conv2d_flops(...)` | Input and output channels, kernel size, output height and width, groups | FLOPs of one convolution (integer) | |
| `perf.onnx_conv_profile(path)` | ONNX model file | Per convolution: shape, FLOPs, bytes moved, arithmetic intensity (FLOP/byte) | |
| `report.input_warnings(rgb)` | uint8 RGB frame | Warnings for wrong size, very dark, very bright or near-uniform frames | |
| `cli.resolve_threshold(arg, weights, cost_ratio)` | `auto` or a number; model file; r | τ, read from `threshold.json` for that model and r | `ValueError` if outside [0, 1] or none stored; `FileNotFoundError` if the file is missing |
| `cli.main(argv)` | `inspect <files or folders> [--weights] [--threshold] [--cost-ratio] [--out]` | Per-frame JSON and PNG, batch summary and HTML index; exit code 0, 1 (bad arguments) or 2 (an input was an ERROR) | Bad files become ERROR reports and the batch continues |

## Appendix C: Additional figures

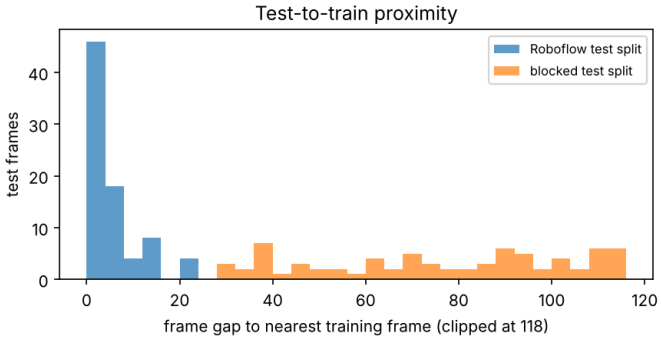



*Figure C1. Gap from each test frame to its nearest training frame. Every provided test frame sits within 30 frames of a training frame; the blocked split removes this.*

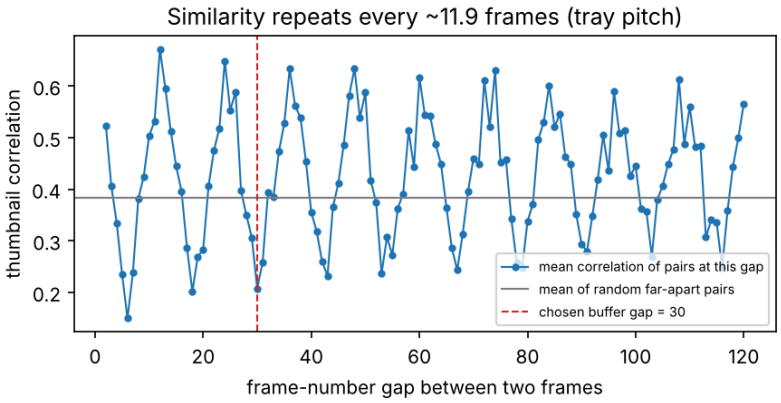



*Figure C2. Correlation of frame thumbnails against frame gap. The curve repeats every 11.9 frames, the spacing between trays;*

*the 30-frame buffer spans about 2.5 trays.*

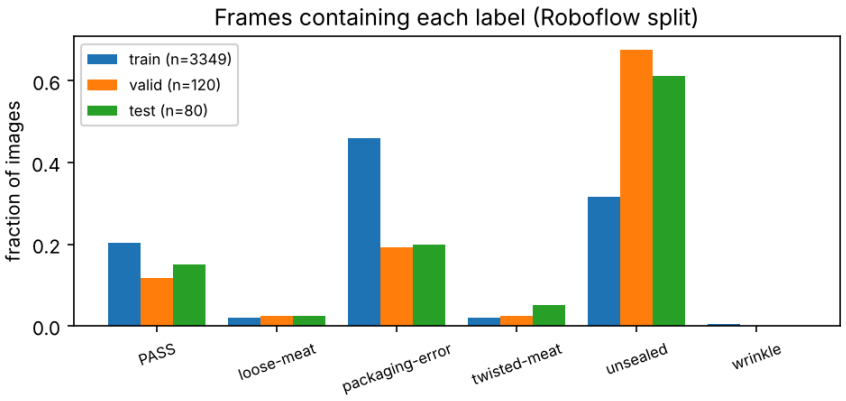



*Figure C3. Share of frames containing each label in the provided splits. PASS is a minority and three classes are rare.*

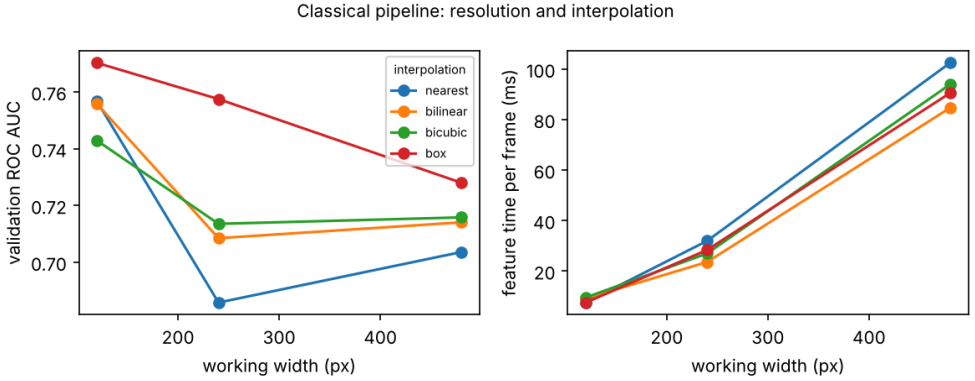



*Figure C4. Classical pipeline: validation AUC (left) and feature time (right) against working width and interpolation method. Box*

*filtering is best at every size; time grows with pixel count.*

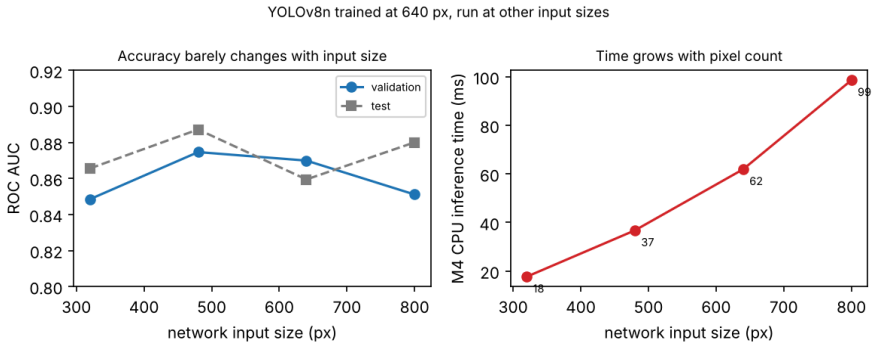



*Figure C5. YOLOv8n trained at 640 px and run at other input sizes: accuracy barely changes, while time grows with pixel count.*

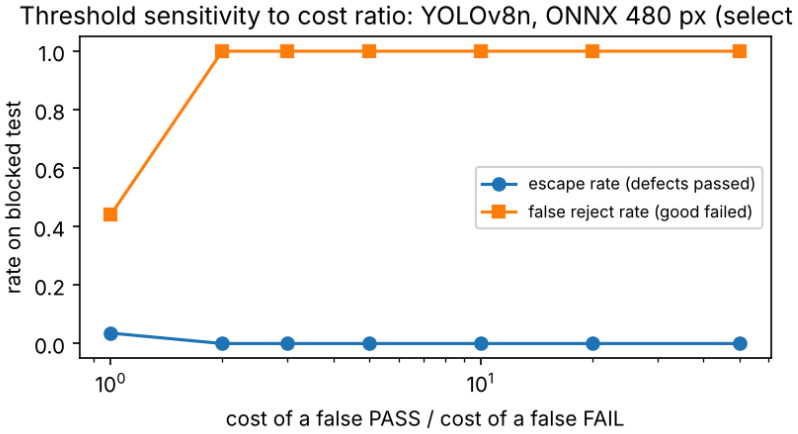



*Figure C6. Selected detector: test escape rate and false reject rate at the threshold chosen on validation for each cost ratio. Only at r = 1 does the threshold pass any frames.*

## Appendix D: Chosen settings

| Model | Settings chosen by grouped cross-validation |
|---|---|
| Colour-histogram logistic regression | C = 0.01, balanced class weights |
| Logistic regression | C = 0.001, balanced class weights |
| Decision tree | maximum depth 5, at least 20 samples per leaf (18 leaves) |
| SVM | RBF kernel, C = 3, gamma = 1e-4 (1,606 support vectors) |
| MLP | one hidden layer of 64 units, L2 penalty alpha = 1.0, early stopping |
| YOLOv8n | 640 px input, up to 60 epochs, patience 20, batch 16, seed 0, COCO-pretrained; stopped at epoch 39 with the weights from epoch 19; deployed at a 480 px input |
| Decision threshold | chosen on validation for each cost ratio; r = 10 by default |

## Appendix E: Verification tests

| Test file | Tests | What they establish |
|---|---|---|
| `test_dataset.py` | 10 | Label files are parsed and malformed ones rejected with the file and line; augmented copies group to one source frame |
| `test_splits.py` | 6 | Whole blocks stay together, no test frame lies within the gap of a training frame, evaluation uses one copy per frame, the split is reproducible |
| `test_decision_evaluate.py` | 8 | The PASS/FAIL rule; the threshold search equals a brute-force search and never rises as the cost ratio rises; Wilson intervals match known values |
| `test_features_detector.py` | 13 | Feature vectors have a fixed length; letterbox, IoU and NMS give known answers; the decoder recovers a hand-encoded box; the FLOP formula matches a loop count; the optimised histogram and HSV conversion match the references |
| `test_onnx_pipeline.py` | 4 | A small synthetic ONNX network runs end to end in fp32 and fp16, and the graph FLOP profiler matches the formula |

| Test file | Tests | What they establish |
|---|---|---|
| `test_report_cli.py` | 5 | Reports are written; corrupt, non-image and invalid inputs become ERROR reports without stopping a batch; thresholds are looked up by model and cost ratio |
| Doctests in `src/` | 6 | Worked examples in the docstrings |

## Appendix F: Environment

Development and classical models: Python 3.13, scikit-learn 1.9.1, scikit-image 0.26.0, NumPy 2.5.3, pandas 3.0.5, ONNX Runtime 1.29.0, Pillow 12.3.0. Timing on the target laptop: Apple MacBook with M4 chip, in a 4-core Linux virtual machine with Python 3.10. Training: Google Colab with an NVIDIA T4 GPU, Ultralytics 8.4.185, PyTorch 2.11.0 (CUDA 13.0), Python 3.13.